

**Laporan Final Project Pemrograman Jaringan**  
*NetBeat: Game Berbasis Rythm*



Bara Semangat Rohmani (5025241144)  
Naufaldi Faqih Abimanyu (5025241184)

**DEPARTEMEN TEKNIK INFORMATIKA  
FAKULTAS TEKNOLOGI DAN INFORMATIKA CERDAS  
INSTITUT TEKNOLOGI SEPULUH NOPEMBER  
SURABAYA  
2026**

## **BAB 1**

### **PENDAHULUAN**

Implementasi sistem *multiplayer real-time* memiliki tantangan teknis yang signifikan, terutama pada sisi arsitektur jaringan dan sinkronisasi *state* antarpemain. Setiap paket data yang dikirimkan oleh komputer pengguna (*client*) harus diproses, disimpan, dan disebar kembali oleh *server* kepada pemain lain secara cepat dan berurutan, agar masing-masing pemain dapat melihat kondisi permainan lawan secara konsisten dan tanpa keterlambatan yang signifikan.

Tantangan ini menjadi relevan pada game bertipe *rhythm/dance* seperti AyoDance, di mana dua pemain bermain secara bersamaan, tetapi terhubung melalui jaringan yang berbeda. Meskipun perhitungan skor dan akurasi ketukan (Perfect/Great/Miss) masing-masing pemain dihitung secara lokal pada perangkatnya sendiri, kualitas pengalaman bermain tetap sangat dipengaruhi oleh kecepatan dan keandalan transmisi data jaringan. Latensi yang tinggi dapat menyebabkan tampilan status lawan (skor, animasi, dan umpan balik ketukan) yang diterima menjadi tidak *real-time*, sehingga mengurangi rasa kompetisi dan interaksi antarpemain. Oleh karena itu, arsitektur jaringan yang dirancang dengan baik diperlukan agar distribusi *state* antar-*client* dapat berjalan secara cepat, stabil, dan tetap terjaga meskipun terjadi gangguan koneksi pada salah satu pemain.

## **BAB 2**

### **DESKRIPSI DAN TUJUAN PROJECT**

#### **2.1 Deskripsi Project**

NetBeat adalah aplikasi game *multiplayer* berbasis jaringan (*network-driven multiplayer rhythm game*) yang dirancang untuk menguji akurasi ketukan, ketepatan waktu, dan kecepatan respons pemain secara *real-time* melalui koneksi *client-server*. Game ini mengambil inspirasi dari genre *rhythm/dance* seperti AyoDance, yang masih populer dimainkan secara berkelompok karena *gameplay*-nya yang sederhana tetapi kompetitif.

Dalam NetBeat, dua pemain (Host/P1 dan Guest/P2) terhubung ke *server* yang sama melalui sebuah room, kemudian masing-masing menjalankan panggung dansa interaktif berisi rangkaian panah 8 arah yang muncul secara dinamis sesuai irama lagu yang dipilih. Setiap pemain menekan tombol *Spacebar* pada area target (*Space Zone*) yang ditentukan, dan hasil ketukan tersebut dikalkulasikan secara lokal ke dalam tiga tingkat akurasi: Perfect Hit, Great Hit, dan Miss. Hasil skor dan status animasi masing-masing pemain kemudian dikirimkan ke *server* untuk disebar ke pemain lawan secara *real-time*, sehingga kedua pemain dapat melihat progres permainan satu sama lain selama sesi berlangsung.

Selain *gameplay* inti, NetBeat juga menyediakan fitur pendukung berupa sistem *room* (pilihan kamar, lagu, dan tingkat kesulitan oleh Host), *lobby* dengan status *ready* kedua pemain, indikator latensi (*ping*) secara *real-time*, fitur *quick chat* singkat antarpemain, serta penanganan *disconnect* agar *server* tetap berjalan stabil ketika salah satu pemain keluar dari permainan secara tiba-tiba.

## 2.2 Tujuan Project

Pada *project* ini, tujuan yang ingin dicapai meliputi:

1. Merancang arsitektur *Client-Server* terpusat berbasis *multi-threading* menggunakan bahasa Python, di mana server menangani setiap koneksi *client* pada *thread* terpisah (`handle_client`) sehingga proses I/O bersifat *non-blocking* dan dapat melayani beberapa *room* serta pemain secara bersamaan.
2. Membangun fitur indikator latensi jaringan (*ping*) berbasis mekanisme Heartbeat (PING-PONG), yang dikirim secara periodik dari *client* setiap 2 detik untuk memantau performa koneksi terhadap *server* secara *real-time*.
3. Menerapkan sistem *logging* otomatis pada berkas `server_network.log` untuk merekam kronologi aktivitas server, meliputi koneksi/diskoneksi pemain, perubahan konfigurasi *room* (lagu dan tingkat kesulitan), serta status *ready* setiap pemain sebelum permainan dimulai.
4. Merancang mekanisme penanganan *disconnect* (*cleanup session*) pada sisi *server*, sehingga ketika salah satu *client* terputus secara tiba-tiba, *server* tidak mengalami *crash* dan slot pemain yang terputus dapat dibersihkan serta diinformasikan kepada pemain lawan melalui paket `OPPONENT_DISCONNECTED`.

## BAB 3 ARSITEKTUR SISTEM

Aplikasi NetBeat mengimplementasikan model arsitektur **Client-Server Terpusat (Dedicated Authority)** yang berjalan di atas protokol transport TCP/IP. Dalam arsitektur ini, server bertindak sebagai otoritas tunggal yang memegang seluruh state permainan (konfigurasi *room*, status *ready* pemain, dan state *gameplay* terbaru), sementara *client* bertugas menangani input pemain, rendering grafis, dan audio secara lokal.

### 3.1 Dedicated Game Server (`server.py`)

*Server* dirancang menggunakan pendekatan *multi-threading*: setiap koneksi *client* baru yang diterima pada `accept()` *loop* utama akan dialokasikan ke sebuah *room* (`ROOM_01–ROOM_03`) berdasarkan slot P1/P2 yang masih kosong, kemudian ditangani oleh *thread* independen melalui fungsi `handle_client`. Dengan pendekatan ini, *server* dapat menampung hingga 3 *room* secara bersamaan (maksimal 6 pemain), dan setiap *thread* melakukan operasi `recv()` secara independen tanpa saling memblokir.

Selain alokasi awal, *server* juga mendukung **perpindahan room secara dinamis (Room Transfer)**. Jika seorang pemain mengirim `ROOM_SETUP` dengan target *room* yang berbeda dari *room* saat ini, *server* akan: melepas slot pemain dari *room* lama, memberi tahu pemain lain di *room* lama melalui `OPPONENT_DISCONNECTED`, kemudian menempatkan pemain pada *room* baru dengan `player_id` (P1/P2) yang ditentukan ulang sesuai slot kosong, dan mengirim ulang `INIT_PLAYER` agar *client* memperbarui identitasnya. Proses ini dicatat ke `server_network.log` sebagai event `ROOM_TRANSFER`.

State permainan disimpan dalam struktur data global `rooms_data`, yang berisi konfigurasi *room* (lagu dan kecepatan), status *ready* masing-masing pemain, *state gameplay* (skor, animasi, *feedback*, *chat*), serta koneksi *socket* aktif tiap pemain. Setiap kali *server* menerima paket `GAME_UPDATE` dari

salah satu *client*, *server* memperbarui `rooms_data` lalu langsung mem-*broadcast snapshot state* terbaru (`REALTIME_STATE`) ke seluruh koneksi aktif dalam room yang sama melalui fungsi `broadcast_to_room`.

### 3.2 Game Client (`main.py`, `scenes.py`, `gameplay.py`)

Di sisi *client*, aplikasi dipisah ke dalam beberapa modul: `main.py` sebagai *entry point* dan *game loop* utama, `scenes.py` untuk *rendering* tampilan menu, konfigurasi *room*, dan *lobby*, serta `gameplay.py` untuk logika dan *rendering* panggung permainan inti.

*Client* menjalankan dua *thread* utama:

- **Main Thread (*Game Loop*)**: berjalan konstan pada 60 FPS (`clock.tick(60)`), menangani pembacaan input *keyboard/mouse*, perpindahan antar-*scene* (Main Menu, Create Room, Lobby, Gameplay), serta *rendering* grafis melalui Pygame.
- **Network Receiver Thread**: dijalankan sekali di awal program (`network_receiver`), berjalan terus-menerus di latar belakang melakukan `recv()` data JSON dari *server* selama aplikasi berjalan. *Thread* ini memperbarui variabel global seperti `my_network_id`, `network_states` (skor dan status lawan), `current_latency`, serta status sinkronisasi *room* (`ROOM_SYNC`), tanpa mengganggu jalannya *rendering* pada *thread* utama.

Pemisahan ini memastikan bahwa proses menerima data dari *server* tidak memblokir *game loop*, sehingga animasi dan input pemain tetap responsif meskipun terjadi keterlambatan jaringan.

## BAB 4 DESAIN PROTOKOL APLIKASI

### 4.1 Pemilihan Protokol Transport dan Alasan

NetBeat menggunakan protokol **TCP** sebagai protokol transport untuk seluruh komunikasi antara *client* dan *server*. Pemilihan ini didasarkan pada beberapa pertimbangan yang disesuaikan dengan karakteristik gameplay NetBeat:

Pertama, seluruh data yang dipertukarkan bersifat **diskrit dan berbasis event** (dikirim setiap terjadi penekanan tombol atau perubahan status), bukan stream data kontinu bervolume tinggi seperti pada game FPS/aksi real-time dengan update per-frame. Berdasarkan pengukuran ukuran paket aktual yang dicatat server, satu paket `GAME_UPDATE` berukuran sekitar 100 byte, dan satu paket `REALTIME_STATE` (berisi state dua pemain) berukuran sekitar 196 byte. Pada skala data sekecil ini, overhead TCP (acknowledgement dan retransmission) tidak signifikan terhadap pengalaman bermain.

Kedua, **keandalan data lebih krusial daripada kecepatan mentah** untuk jenis data yang dikirim NetBeat. Paket seperti `ROOM_SETUP`, `PLAYER_READY`, dan `GAME_UPDATE` (yang membawa skor akumulatif) tidak boleh hilang di tengah jalan. Kehilangan satu paket `GAME_UPDATE` berarti skor pemain tidak akan pernah ter-update di sisi lawan untuk ketukan tersebut, dan tidak ada mekanisme koreksi otomatis (sistem bersifat *stateless update*, bukan *delta yang dapat di-replay*). Jika menggunakan UDP, paket yang hilang akan menyebabkan ketidaksinkronan skor permanen tanpa cara memperbaikinya kecuali menambah mekanisme acknowledgement dan retry secara manual yang pada dasarnya berarti membangun ulang sebagian dari apa yang sudah disediakan TCP secara native.

Ketiga, koneksi TCP bersifat **connection-oriented**, sehingga server dapat mendeteksi disconnect pemain secara langsung melalui `recv()` yang mengembalikan data kosong atau exception (`ConnectionResetError`), tanpa perlu membangun mekanisme timeout/heartbeat tambahan khusus untuk deteksi disconnect (heartbeat PING-PONG pada NetBeat digunakan murni untuk pengukuran latency, bukan deteksi koneksi putus).

Dengan pertimbangan tersebut, TCP dipilih karena kebutuhan NetBeat (keandalan pengiriman state pada volume data yang relatif rendah) lebih dipentingkan dibanding kecepatan maksimal yang ditawarkan UDP, yang trade-off-nya (paket bisa hilang/tidak berurutan) baru benar-benar bernilai pada game dengan update posisi/state berfrekuensi sangat tinggi (puluhan hingga ratusan kali per detik).

#### 4.2 Format Pesan (*Message Dictionary*)

Protokol lapisan aplikasi NetBeat dirancang secara mandiri (*custom protocol*) menggunakan format **JSON**. Setiap pesan di-*encode* ke UTF-8 sebelum dikirim (`json.dumps(...).encode('utf-8')`) dan di-*decode* serta di-*parse* kembali menjadi dictionary Python di sisi penerima (`json.loads(data.decode('utf-8'))`). Setiap pesan wajib memiliki key "**type**" sebagai pengidentifikasi jenis instruksi.

#### 4.3 Daftar Tipe Pesan

| Tipe Pesan   | Arah                 | Keterangan                                                                                                                                                                              |
|--------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INIT_PLAYER  | Server → Client      | Dikirim saat koneksi TCP berhasil dibangun atau setelah room transfer, memberikan identitas <code>player_id</code> (P1/P2) kepada client.                                               |
| ROOM_SETUP   | Client (P1) → Server | Dikirim oleh Host untuk menentukan lagu dan kecepatan room, atau untuk berpindah ke room lain. Jika room tujuan berbeda dari room saat ini, server memproses ini sebagai Room Transfer. |
| ROOM_SYNC    | Server → Client      | Diteruskan server ke seluruh pemain dalam room setelah <code>ROOM_SETUP</code> diterima, menyamakan konfigurasi lagu dan kecepatan di sisi Guest.                                       |
| PLAYER_READY | Client → Server      | Menandakan status siap suatu pemain. Jika kedua pemain ready, server membalas                                                                                                           |
| START_MATCH  | Server → Client      | Sinyal bagi kedua client untuk memulai sesi gameplay secara bersamaan.                                                                                                                  |
| GAME_UPDATE  | Client → Server      | Dikirim setiap terjadi event ketukan (~100 byte/paket), membawa skor tambahan, status animasi, feedback, dan pesan quick chat.                                                          |

|                       |                 |                                                                                                 |
|-----------------------|-----------------|-------------------------------------------------------------------------------------------------|
| REALTIME_STATE        | Server → Client | Snapshot state seluruh pemain dalam room (~196 byte/paket), di-broadcast setiap server menerima |
| PING / PONG           | Client → Server | Mekanisme heartbeat untuk mengukur RTT, dikirim client setiap 2 detik.                          |
| OPPONENT_DISCONNECTED | Server → Client | Dikirim ke pemain yang tersisa ketika lawan terputus atau berpindah room.                       |
| SERVER_FULL           | Server → Client | Dikirim saat seluruh room (ROOM_01–03) penuh, sebelum koneksi ditutup.                          |

## BAB 5

### PENGUJIAN PERFORMA DAN BEBAN SERVER

#### 5.1 Pengujian Latensi (Heartbeat PING-PONG)

Pengujian latensi dilakukan dengan mekanisme heartbeat PING-PONG, di mana client mengirim paket **PING** berisi timestamp setiap 2 detik, dan server membalas dengan **PONG** membawa timestamp yang sama sehingga client dapat menghitung selisih waktu pulang-pergi (RTT). Server juga mencatat estimasi RTT ini ke `server_network.log` setiap kali heartbeat diterima.

Pengujian dilakukan dengan menjalankan server dan kedua client (P1 dan P2) pada perangkat yang sama (localhost, `127.0.0.1`). Hasil pengukuran menunjukkan nilai **EST\_LATENCY** sebesar 0 ms secara konsisten untuk kedua pemain. Hal ini sesuai dengan ekspektasi pengujian pada loopback interface, di mana paket tidak melewati jaringan fisik sehingga waktu transmisi berada di bawah resolusi pengukuran (milidetik). **Pengujian ini mengonfirmasi bahwa mekanisme heartbeat berfungsi dengan benar (paket PING berhasil dikirim, diterima, dan dibalas), namun belum merepresentasikan kondisi latensi pada jaringan nyata (LAN/WAN), karena seluruh komunikasi terjadi dalam satu perangkat tanpa melalui medium jaringan fisik.**

#### 5.2 Pengujian Beban Server (Multi-Room)

Pengujian fungsional multi-room dilakukan dengan menjalankan lebih dari satu room secara bersamaan (lebih dari satu pasang client terhubung ke room yang berbeda). Hasil pengujian menunjukkan server dapat menangani beberapa room secara paralel tanpa terjadi crash, kesalahan alokasi `player_id`, maupun pencampuran data state antar-room — setiap room tetap mempertahankan state (**states**, **players\_ready**, **config**) secara independen sesuai struktur `rooms_data`.

Pengujian ini bersifat **fungsional** (memastikan fitur multi-room bekerja sebagaimana dirancang) dan belum mencakup **pengujian beban kuantitatif** (misalnya mengukur penggunaan CPU/RAM server seiring bertambahnya jumlah room/client aktif, atau menguji jumlah room mendekati batas maksimum 3 room/6 client secara bersamaan dengan beban gameplay penuh).

### 5.3 Pengujian Ukuran Payload

Berdasarkan pencatatan ukuran paket pada `server_network.log`, diperoleh contoh ukuran paket sebagai berikut:

| Tipe Paket     | Arah                        | Ukuran (contoh) |
|----------------|-----------------------------|-----------------|
| GAME_UPDATE    | Client → Server             | ~100 Bytes      |
| REALTIME_STATE | Server → Client (broadcast) | ~196 Bytes      |

Ukuran `REALTIME_STATE` yang kurang lebih dua kali lipat `GAME_UPDATE` sesuai dengan struktur datanya: `REALTIME_STATE` membawa state kedua pemain (P1 dan P2) dalam satu payload, sedangkan `GAME_UPDATE` hanya membawa data satu pemain pengirim. Ukuran paket pada kisaran ratusan byte ini mendukung argumen pemilihan TCP pada bagian 4.1, karena overhead protokol TCP tidak menjadi signifikan pada volume data sekecil ini.

## BAB 6 HASIL DAN ANALISIS

### 6.1 Hasil Pengujian Fungsional

Berdasarkan hasil pengujian, seluruh fitur wajib pada arsitektur Client-Server NetBeat berjalan sesuai rancangan tanpa indikasi kegagalan parsing JSON maupun crash server selama pengujian:

- Mekanisme **matchmaking dan sinkronisasi room** berjalan dengan baik: ketika Host (P1) mengonfigurasi lagu dan kecepatan melalui `ROOM_SETUP`, server meneruskan konfigurasi tersebut ke Guest (P2) melalui `ROOM_SYNC`, sehingga kedua client memiliki state lobby yang identik sebelum pertandingan dimulai. Sistem **Room Transfer** juga berfungsi — pemain dapat berpindah room secara dinamis, dengan server memperbarui alokasi `player_id` dan memberi tahu pemain di room lama melalui `OPPONENT_DISCONNECTED`.
- Real-time update state** selama gameplay berjalan sesuai desain: setiap event `GAME_UPDATE` dari satu client langsung memicu broadcast `REALTIME_STATE` ke seluruh pemain di room yang sama, sehingga skor, status animasi (dancing/miss/idle), dan feedback ketukan (Perfect/Great/Miss) lawan terlihat ter-update di sisi client lain.

### 6.2 Analisis Data Performa

Mengacu pada data BAB 5, dapat disampaikan beberapa analisis:

- Ukuran paket `GAME_UPDATE` (~100 byte) dan `REALTIME_STATE` (~196 byte) berada pada kisaran yang sangat kecil dibanding MTU jaringan umum (~1500 byte), sehingga setiap paket NetBeat masih jauh di bawah ambang fragmentasi. Kombinasi ukuran paket kecil dan frekuensi pengiriman berbasis event (bukan per-frame) berarti **beban jaringan aktual NetBeat relatif ringan**, bahkan jika dijalankan pada koneksi dengan bandwidth terbatas.

- b. Hasil pengujian latensi (5.1) yang menunjukkan `EST_LATENCY = 0 ms` pada localhost **tidak dapat dijadikan kesimpulan performa jaringan**, karena tidak ada medium jaringan fisik yang dilalui. Nilai ini hanya mengonfirmasi bahwa siklus PING-PONG berjalan tanpa error pada level kode. Implikasinya, klaim "sistem memiliki latensi rendah" **belum dapat dibuktikan** untuk kondisi LAN/WAN, dan ini menjadi batasan utama pengujian pada laporan ini (lihat 8.2 Saran).
- c. Pengujian fungsional multi-room (5.2) menunjukkan bahwa struktur data `rooms_data` berhasil mengisolasi state antar-room, yang relevan terhadap rubrik **Stabilitas Sistem (reliability dan concurrency handling)** — meskipun pengujian ini belum diuji pada beban tinggi (banyak room aktif dengan gameplay penuh secara simultan).

## BAB 7

### KENDALA DAN SOLUSI

#### 7.1 Sinkronisasi Skor pada Intensitas Tinggi

*Kendala:* Pada rancangan awal, data permainan (skor dan feedback ketukan) dikirim murni berdasarkan event tanpa snapshot terpusat. Ketika intensitas ketukan meningkat pada lagu berkecepatan tinggi (Hard Difficulty), pertukaran paket sering mengalami tabrakan data (*data race*) atau penumpukan antrean pada buffer socket, yang menyebabkan skor akhir antar-pemain menjadi tidak sinkron.

*Solusi:* Arsitektur diubah menjadi **State-Driven Client-Server**: setiap kali client mendeteksi penekanan tombol, client mengirim status lokal singkat (`GAME_UPDATE`) ke server. Server memproses dan memperbarui state global (`rooms_data`), lalu langsung membalas dengan snapshot status seluruh pemain (`REALTIME_STATE`) ke semua client di room tersebut. Dengan ini, server menjadi satu-satunya sumber kebenaran (*single source of truth*) untuk state permainan, sehingga tidak ada client yang menyimpan state lawan secara independen yang bisa berbeda satu sama lain.

#### 7.2 Server Crash Saat Client Disconnect Mendadak

*Kendala:* Ketika salah satu pemain menutup jendela Pygame secara mendadak (misalnya dengan tombol `close [X]`), `server.py` mengalami crash total dengan error `ConnectionResetError: [WinError 10054] An existing connection was forcibly closed by the remote host`. Crash ini menyebabkan seluruh server mati, sehingga pemain lain yang berada di room berbeda juga ikut terputus.

*Solusi:* Fungsi `handle_client` diperkuat dengan membungkus seluruh loop `recv()` dalam blok `try-except` yang menangkap exception koneksi (termasuk `ConnectionResetError`). Ketika exception tertangkap, server tidak mati, melainkan langsung menjalankan proses pembersihan (*cleanup session*): slot koneksi pemain dikosongkan, status ready di-reset, state pemain dikembalikan ke kondisi awal (`IDLE`), dan pemain lain di room tersebut diberi tahu melalui `OPPONENT_DISCONNECTED`. Operasi `sendall` pada `broadcast_to_room` juga dibungkus try-except per koneksi, sehingga kegagalan pengiriman ke satu client tidak menggagalkan broadcast ke client lain.



## BAB 8

### KESIMPULAN DAN SARAN

#### 8.1 Kesimpulan

Berdasarkan tahapan perancangan, implementasi, dan pengujian yang telah dilakukan pada NetBeat, dapat ditarik kesimpulan sebagai berikut:

1. Arsitektur **Client-Server Terpusat berbasis multi-threading** berhasil dibangun menggunakan Python socket dan berjalan stabil dalam menangani matchmaking, sinkronisasi room, perpindahan room dinamis (Room Transfer), dan distribusi state real-time antar-pemain.
2. Protokol **TCP** dipilih karena karakteristik data NetBeat, paket berukuran kecil (~100-196 byte), dikirim berbasis event, dan membutuhkan keandalan pengiriman (skor tidak boleh hilang), lebih sesuai dengan jaminan *connection-oriented* dan *reliable delivery* dari TCP, dibanding kebutuhan kecepatan ekstrem yang ditawarkan UDP namun tidak relevan pada skala data ini.
3. Mekanisme **penanganan disconnect** (cleanup session dan try-except pada operasi socket) berhasil mencegah crash server ketika salah satu client terputus secara tiba-tiba, sehingga pemain lain di room berbeda tidak terdampak.
4. Pengujian yang dilakukan (latensi heartbeat dan multi-room) **bersifat fungsional** dan dilakukan pada lingkungan localhost, sehingga membuktikan kebenaran logika sistem namun belum merepresentasikan kondisi jaringan dan beban nyata.

#### 8.2 Saran

Untuk pengembangan NetBeat selanjutnya, penulis merumuskan beberapa saran:

1. **Pengujian pada jaringan nyata (LAN/WAN):** melakukan pengujian latensi dan stabilitas dengan client dan server berjalan pada perangkat berbeda yang terhubung melalui jaringan fisik, untuk memperoleh data RTT yang representatif.
2. **Pengujian beban kuantitatif:** mengukur penggunaan CPU dan memori server seiring bertambahnya jumlah room dan client aktif secara bersamaan, termasuk pada kondisi mendekati kapasitas maksimum (3 room/6 client).
3. **Migrasi protokol hybrid:** untuk pengembangan ke arah game dengan update posisi berfrekuensi tinggi (misalnya animasi pose real-time per frame), dapat dipertimbangkan protokol UDP untuk data tersebut, sementara TCP tetap dipertahankan untuk fungsi krusial seperti matchmaking, autentikasi, dan rekap skor akhir.